ỦY BAN NHÂN DÂN THÀNH PHỐ HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC SÀI GÒN

KHOA CÔNG NGHỆ THÔNG TIN

CÁC CÔNG NGHỆ LẬP TRÌNH HIỆN ĐẠI

Phân tích kiến trúc và triển khai hệ thống thương mại điện tử ProjectWeb

Giảng viên hướng dẫn:	**TS Đỗ Như Tài**
Sinh viên :	**Mssv:**
Nguyễn Việt Đức	3122411044
Nguyễn Huỳnh Phương Lộc	3122411117
Nguyễn Phạm Hoàng Quân	3122411169
Nguyễn Trần Quốc Tiến	3122411208

Thành phố Hồ Chí Minh, tháng 4 năm 2026

LỜI CẢM ƠN

Em xin chân thành cảm ơn giảng viên phụ trách môn học đã hướng dẫn, định hướng và tạo điều kiện để em tiếp cận kiến trúc microservices thông qua bài tập thực hành. Trong quá trình thực hiện báo cáo này, em không chỉ tìm hiểu cấu trúc mã nguồn của dự án mà còn phân tích cách các dịch vụ giao tiếp, cách triển khai và tư duy tổ chức hệ thống theo hướng hiện đại.

Em cũng xin cảm ơn các tài liệu tham khảo về Spring Boot, Spring Cloud, Docker, Kubernetes, Kafka, MongoDB, Redis, React và các nguồn hướng dẫn liên quan đã giúp em có cơ sở đối chiếu khi đánh giá hệ thống. Báo cáo được xây dựng với mục tiêu học tập và phân tích kỹ thuật; vì vậy chắc chắn vẫn còn những điểm chưa thật hoàn thiện. Em rất mong nhận được góp ý của giảng viên để tiếp tục hoàn thiện kiến thức về thiết kế hệ thống phân tán.

LỜI MỞ ĐẦU

Trong bối cảnh các ứng dụng web ngày càng mở rộng cả về quy mô người dùng lẫn độ phức tạp nghiệp vụ, kiến trúc microservices trở thành một hướng tiếp cận được nhiều tổ chức lựa chọn nhờ khả năng tách nhỏ hệ thống, cho phép triển khai độc lập, mở rộng linh hoạt và bảo trì hiệu quả hơn so với mô hình nguyên khối truyền thống.

Bài tập 6 trong repository GitHub được lựa chọn phân tích trong báo cáo này là một hệ thống thương mại điện tử gồm frontend React và backend microservices xây dựng bằng Spring Boot. Từ cấu trúc thư mục, README, tập tin cấu hình và các thành phần triển khai, có thể nhận thấy dự án không chỉ dừng ở mức chia service, mà còn tích hợp các thành phần hạ tầng quan trọng như API Gateway, Eureka Server, Kafka, Redis, MongoDB, Docker Compose, Kubernetes, Zipkin và Prometheus.

Báo cáo tập trung phân tích kiến trúc hệ thống, chức năng từng service, mô hình giao tiếp, dữ liệu, cách triển khai và đánh giá ưu nhược điểm của giải pháp. Ở những phần không đọc trực tiếp được toàn bộ mã nguồn chi tiết, nội dung được trình bày theo hướng phân tích kiến trúc dựa trên bằng chứng từ cấu trúc dự án và thông lệ triển khai tương ứng, đồng thời nêu rõ đó là nhận định phân tích thay vì khẳng định tuyệt đối.

MỤC LỤC

LỜI CẢM ƠN 2
LỜI MỞ ĐẦU 3
DANH MỤC HÌNH 6
CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI 7
1.1. Giới thiệu đề tài 7
1.2. Mục tiêu nghiên cứu 7
1.3. Phạm vi và đối tượng nghiên cứu 7
1.4. Phương pháp thực hiện 8
1.5. Ý nghĩa của đề tài 8
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VỀ MICROSERVICES 8
2.1. Kiến trúc phần mềm và sự tiến hóa từ monolithic 8
2.2. Khái niệm microservices 9
2.3. Đặc điểm của kiến trúc microservices 9
2.4. Ưu điểm của microservices 9
2.5. Nhược điểm và thách thức 9
2.6. Các thành phần hạ tầng thường gặp 10
2.7. So sánh monolithic và microservices 10
CHƯƠNG 3. PHÂN TÍCH DỰ ÁN PROJECTWEB-MAIN 10
3.1. Khái quát dự án 10
3.2. Cấu trúc thư mục chính 11
3.3. Phân tích frontend 11
3.4. Phân tích backend và các service 12
3.5. Vai trò từng service trong nghiệp vụ 13
3.6. Luồng nghiệp vụ chính của hệ thống 14
3.7. Nhận xét về mức độ hoàn thiện của dự án 14
CHƯƠNG 4. THIẾT KẾ KIẾN TRÚC MICROSERVICES CỦA HỆ THỐNG 15
4.1. Nguyên tắc phân tách miền nghiệp vụ 15
4.2. API Gateway 16

4.3. Service Discovery với Eureka 16
4.4. Giao tiếp đồng bộ và bất đồng bộ 17
4.5. Quản lý dữ liệu 17
4.6. Bảo mật và xác thực 18
4.7. Quan sát hệ thống và truy vết 18
CHƯƠNG 5. TRIỂN KHAI, VẬN HÀNH VÀ CÔNG NGHỆ SỬ DỤNG 18
5.1. Stack công nghệ của dự án 18
5.2. Triển khai local với Docker Compose 19
5.3. CI/CD và container hóa 20
5.4. Triển khai trên Kubernetes 20
5.5. Tính nhất quán tài liệu và cấu hình 21
CHƯƠNG 6. ĐÁNH GIÁ HỆ THỐNG 22
6.1. Ưu điểm 22
6.2. Hạn chế 23
6.3. Đánh giá theo tiêu chí kiến trúc 23
6.4. Phân tích rủi ro kỹ thuật 24
CHƯƠNG 7. HƯỚNG PHÁT TRIỂN 24
7.1. Hoàn thiện tài liệu kỹ thuật 24
7.2. Tăng cường bảo mật 24
7.3. Tăng độ bền vững của hệ thống 25
7.4. Cải tiến dữ liệu và hiệu năng 25
7.5. Bổ sung kiểm thử và quản trị API 25
KẾT LUẬN 26
TÀI LIỆU THAM KHẢO 27

DANH MỤC HÌNH

Hình 1. Minh họa giao diện dự kiến dựa trên cấu trúc frontend

Hình 2. Kiến trúc tổng thể dự án microservices

Hình 3. Luồng xử lý đặt hàng điển hình

Hình 4 . Kiến trúc tổng thể hệ thống ProjectWeb-main

Hình 5. Phân tách miền nghiệp vụ theo bounded context

Hình 6 . Chuỗi xử lý đặt hàng và hậu xử lý sự kiện

Hình 7. Mô hình triển khai và vận hành

Hình 8: trang chủ của trang web

Hình 9. Biểu đồ đánh giá mức sẵn sàng kiến trúc

CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI

1.1. Giới thiệu đề tài

Đề tài của báo cáo là phân tích một dự án microservices phục vụ cho bài toán thương mại điện tử. Đây là dạng hệ thống phù hợp để nghiên cứu kiến trúc phân tán vì nghiệp vụ có nhiều miền chức năng rõ ràng như người dùng, sản phẩm, đơn hàng, thanh toán và thông báo. Khi số lượng chức năng tăng dần, việc tách thành nhiều service giúp hạn chế sự phụ thuộc chặt và giảm rủi ro khi thay đổi.

Đối tượng được phân tích là thư mục ProjectWeb-main trong bài tập 6. Phần frontend được phát triển bằng React, Vite và Tailwind CSS; phần backend được tổ chức thành nhiều service sử dụng Spring Boot và Spring Cloud. Ngoài ra dự án có kèm Docker Compose, cấu hình CI/CD với GitHub Actions và manifest Kubernetes, cho thấy định hướng triển khai thực tế hơn một bài tập CRUD đơn giản.

1.2. Mục tiêu nghiên cứu

- Hiểu được cách một hệ thống thương mại điện tử được chia tách thành các microservice độc lập.
- Phân tích vai trò của các thành phần hạ tầng như API Gateway, Service Discovery, Message Broker, Cache và Observability.
- Đánh giá mức độ phù hợp của kiến trúc microservices đối với bài toán đặt hàng trực tuyến.
- Xác định ưu điểm, hạn chế và các hướng cải tiến cho hệ thống trong tương lai.
- Rèn luyện khả năng đọc cấu trúc dự án, suy luận kiến trúc và trình bày báo cáo kỹ thuật theo chuẩn học thuật.

1.3. Phạm vi và đối tượng nghiên cứu

Phạm vi nghiên cứu của báo cáo không nhằm kiểm thử mọi endpoint hay toàn bộ logic chi tiết trong từng lớp mã nguồn, mà tập trung vào mức kiến trúc và tổ chức hệ thống. Cụ thể, báo cáo phân tích: cấu trúc repo, các module backend, phần frontend, mô hình dữ liệu ở mức khái niệm, cơ chế giao tiếp giữa service, cách triển khai local bằng Docker Compose, cách triển khai cụm thông qua Kubernetes và pipeline CI/CD.

Đối tượng nghiên cứu bao gồm các service: user-service, product-service, order-service, payment-service, notification-service, eureka-server và api-gateway; đồng thời xem xét vai trò của MongoDB, Redis, Kafka, Zipkin và Prometheus trong tổng thể hệ thống.

1.4. Phương pháp thực hiện

- Đọc README của frontend và backend để thu thập thông tin công nghệ, thư mục và cách chạy hệ thống.
- Đọc cấu trúc thư mục của từng module để nhận diện ranh giới service.
- Đọc pom tổng và docker-compose để xác định stack kỹ thuật, phụ thuộc và môi trường chạy.
- Đối chiếu với kiến thức nền tảng về Spring Cloud và kiến trúc microservices để suy luận sơ đồ và luồng xử lý.
- Chuẩn hóa kết quả thành báo cáo học thuật có sơ đồ minh họa, bảng phân tích và đánh giá.

1.5. Ý nghĩa của đề tài

Việc phân tích một dự án microservices hoàn chỉnh giúp sinh viên tiếp cận các vấn đề thực tế của hệ thống hiện đại: gateway, đăng ký dịch vụ, message broker, cache, tracing, triển khai container và tự động hóa build/deploy. Đây là bước chuyển quan trọng từ tư duy viết chương trình đơn lẻ sang tư duy thiết kế hệ thống, nơi tính sẵn sàng, độ mở rộng, khả năng quan sát và tính độc lập triển khai đóng vai trò trung tâm.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VỀ MICROSERVICES

2.1. Kiến trúc phần mềm và sự tiến hóa từ monolithic

Kiến trúc phần mềm là cách tổ chức các thành phần và các mối quan hệ giữa chúng nhằm đáp ứng yêu cầu chức năng lẫn phi chức năng của hệ thống. Trong mô hình monolithic, toàn bộ chức năng thường được đóng gói trong một ứng dụng duy nhất. Cách tiếp cận này có ưu điểm là đơn giản ở giai đoạn đầu, nhưng khi quy mô tăng lên sẽ dễ gặp vấn đề về độ phụ thuộc cao, thời gian build và deploy dài, khó mở rộng từng phần, và rủi ro một lỗi có thể ảnh hưởng toàn bộ hệ thống.

Microservices ra đời như một cách tiếp cận chia hệ thống thành các dịch vụ nhỏ, độc lập và xoay quanh từng miền nghiệp vụ. Mỗi service có thể được triển khai, mở rộng và thậm chí phát triển bởi nhóm khác nhau. Thay vì một ứng dụng khổng lồ, hệ thống trở thành tập hợp các thành phần liên lạc với nhau qua API đồng bộ hoặc qua cơ chế bất đồng bộ như message broker.

2.2. Khái niệm microservices

Microservices là phong cách kiến trúc trong đó một ứng dụng được xây dựng từ nhiều dịch vụ nhỏ, mỗi dịch vụ triển khai một năng lực nghiệp vụ cụ thể, có vòng đời phát triển riêng và giao tiếp với nhau qua mạng. Điểm cốt lõi của microservices không chỉ là tách nhiều project, mà là tách theo bounded context, nghĩa là mỗi service sở hữu rõ ràng một miền trách nhiệm và dữ liệu liên quan.

Ví dụ trong hệ thống thương mại điện tử, thông tin người dùng không nên bị chỉnh sửa trực tiếp bởi service đơn hàng; thay vào đó order-service chỉ gọi hoặc tiêu thụ dữ liệu cần thiết từ user-service và tập trung vào logic đặt hàng. Cách tổ chức này làm tăng tính mô-đun, đồng thời đặt ra yêu cầu thiết kế giao tiếp và quản trị dữ liệu cẩn thận hơn.

2.3. Đặc điểm của kiến trúc microservices

- Mỗi service là một đơn vị triển khai độc lập.
- Mỗi service tập trung vào một năng lực nghiệp vụ tương đối hẹp.
- Dữ liệu thường được sở hữu cục bộ theo service, tránh phụ thuộc trực tiếp vào schema chung duy nhất.
- Giao tiếp giữa các service diễn ra qua HTTP/REST, gRPC hoặc messaging.
- Hệ thống thường cần thêm hạ tầng hỗ trợ như service discovery, API gateway, logging tập trung, tracing và monitoring.
- Khả năng tự động hóa build, test và triển khai là gần như bắt buộc để quản lý độ phức tạp.

2.4. Ưu điểm của microservices

- Dễ mở rộng theo từng chức năng có tải cao.
- Dễ bảo trì và phát triển song song vì ranh giới service rõ ràng.
- Giảm rủi ro khi triển khai thay đổi vì không phải deploy lại toàn bộ hệ thống.
- Cho phép linh hoạt về công nghệ trong từng service nếu cần.
- Phù hợp với hệ thống nghiệp vụ lớn, thay đổi thường xuyên.

2.5. Nhược điểm và thách thức

- Tăng độ phức tạp trong vận hành do có nhiều tiến trình, nhiều cấu hình và nhiều điểm giao tiếp.
- Cần cơ chế quản lý lỗi mạng, timeout, retry, circuit breaker nếu hệ thống lớn hơn.
- Khó đảm bảo nhất quán dữ liệu toàn cục như trong monolithic.
- Việc debug và quan sát lỗi khó hơn do luồng xử lý đi qua nhiều service.
- Đòi hỏi hiểu biết tốt về DevOps, container, orchestration và monitoring.

2.6. Các thành phần hạ tầng thường gặp

Trong hệ microservices, ngoài các service nghiệp vụ còn cần một nhóm hạ tầng hỗ trợ. API Gateway đóng vai trò cửa vào thống nhất cho client, cho phép định tuyến request, xử lý CORS, xác thực tập trung hoặc áp chính sách bảo mật. Service Discovery như Eureka giúp service tìm thấy nhau trong môi trường động. Redis thường dùng làm cache hoặc lưu session tạm thời để giảm tải database. Kafka làm nền tảng truyền thông bất đồng bộ cho các sự kiện nghiệp vụ. Zipkin hỗ trợ distributed tracing để xem request đi qua những service nào, trong khi Prometheus phục vụ thu thập metrics và giám sát hiệu năng.

Những thành phần này đều xuất hiện trong dự án đang phân tích, chứng tỏ tác giả không chỉ xây dựng service đơn lẻ mà đã tiếp cận mô hình microservices tương đối đầy đủ.

2.7. So sánh monolithic và microservices

Tiêu chí	Monolithic	Microservices
Tổ chức mã nguồn	Một khối ứng dụng duy nhất	Nhiều service độc lập
Triển khai	Triển khai toàn bộ ứng dụng	Triển khai theo từng service
Mở rộng	Mở rộng cả hệ thống	Mở rộng đúng service cần thiết
Quản lý dữ liệu	Thường dùng DB tập trung	Có xu hướng phân tách theo service
Độ phức tạp vận hành	Thấp hơn giai đoạn đầu	Cao hơn nhưng linh hoạt hơn
Phù hợp	Ứng dụng nhỏ, ít thay đổi	Hệ thống lớn, nhiều miền nghiệp vụ

CHƯƠNG 3. PHÂN TÍCH DỰ ÁN PROJECTWEB-MAIN

3.1. Khái quát dự án

ProjectWeb-main là một hệ thống web thương mại điện tử được tổ chức thành hai khối lớn: frontend đặt tại FE/react-e-commerce và backend đặt tại BE. README của frontend mô tả đây là phần giao diện người dùng xây dựng bằng React, sử dụng Vite để build và Tailwind CSS để xây dựng giao diện hiện đại. Các tính năng được nêu gồm đăng nhập/đăng ký, giỏ hàng, chi tiết sản phẩm, thanh toán và trang quản trị.

Ở phía backend, README mô tả đây là dự án microservices sử dụng Java, Spring Boot, Eureka, Spring Cloud Gateway, PostgreSQL, Apache Kafka, Docker và Kubernetes. Tuy nhiên khi đọc pom tổng và docker-compose có thể thấy dự án thực tế còn sử dụng MongoDB, Redis, Zipkin và Prometheus; do đó có thể hiểu hệ thống đã phát triển xa hơn mô tả ngắn trong phần giới thiệu.

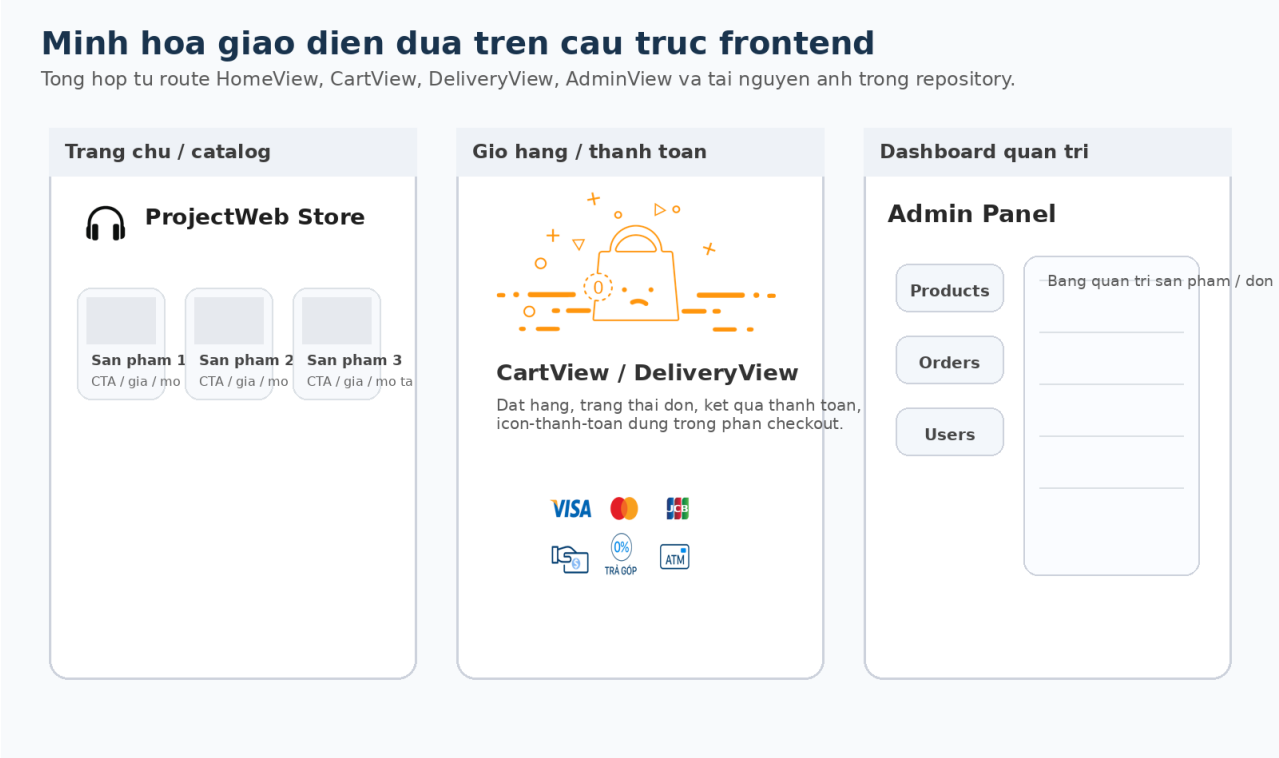
3.2. Cấu trúc thư mục chính

Thành phần	Vị trí	Ý nghĩa
Frontend	FE/react-e-commerce	Giao diện người dùng, route, store, component, view.
Backend services	BE/*-service	Các microservice nghiệp vụ.
Gateway	BE/api-gateway	Điểm vào tập trung cho client.
Service discovery	BE/eureka-server	Đăng ký và tìm kiếm service.
Hạ tầng triển khai	BE/docker-compose.yml	Chạy local nhiều container.
Kubernetes	BE/k8s	Manifest triển khai lên cụm.
CI/CD	BE/.github/workflows	Tự động hóa build và deploy.

3.3. Phân tích frontend

Frontend được tổ chức theo cấu trúc khá điển hình của một ứng dụng React. Thư mục src bao gồm assets, components, helpers, store, views cùng với App.jsx và main.jsx. Điều này cho thấy giao diện được chia thành phần tái sử dụng, phần trợ giúp, phần quản lý trạng thái và các màn hình chính. Việc sử dụng Tailwind CSS giúp tăng tốc độ xây dựng UI, còn Vite giúp rút ngắn thời gian khởi động và build trong quá trình phát triển.

Từ mô tả README, frontend có vai trò tiêu thụ API từ backend thông qua axios và react router. Đây là cách phù hợp với microservices vì client không cần biết trực tiếp mọi service nội bộ; thay vào đó client nên giao tiếp chủ yếu với API Gateway. Bằng cách này, frontend được tách rời khỏi chi tiết định tuyến phía server, giảm độ phụ thuộc vào tên service và topology của backend.

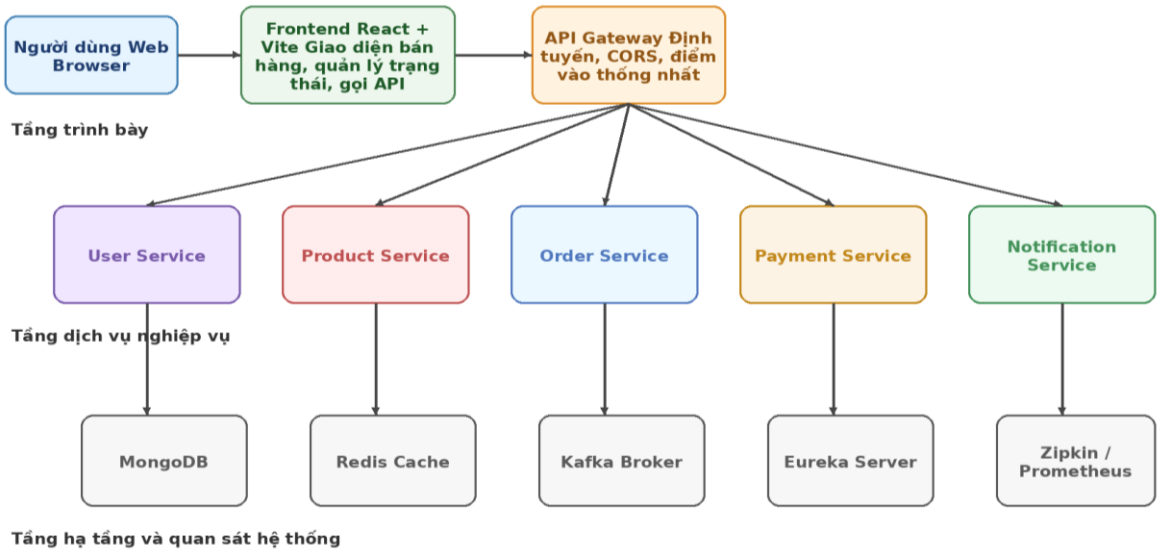


Hình 1. Minh họa giao diện dự kiến dựa trên cấu trúc frontend

3.4. Phân tích backend và các service

Phần backend là trọng tâm của đề tài. Dựa trên cấu trúc thư mục, hệ thống gồm ít nhất 7 module: user-service, product-service, order-service, payment-service, notification-service, eureka-server và api-gateway. Sự hiện diện của parent pom cho thấy đây là mô hình multi-module Maven, rất phổ biến trong các dự án Java lớn. Kiểu tổ chức này giúp quản lý phụ thuộc dùng chung, chuẩn hóa phiên bản thư viện và thống nhất cách build toàn bộ hệ thống.

Từ docker-compose có thể nhận thấy user-service, product-service và order-service đều kết nối với MongoDB, đồng thời product-service và order-service còn kết nối Redis và Kafka. Notification-service dùng Kafka, còn payment-service hoạt động tương đối độc lập hơn. Các dịch vụ đều đăng ký về eureka-server và gateway đóng vai trò public endpoint cho frontend. Ngoài ra hệ thống có Zipkin để trace request và Prometheus để thu thập metrics.



Hình 2. Kiến trúc tổng thể dự án microservices

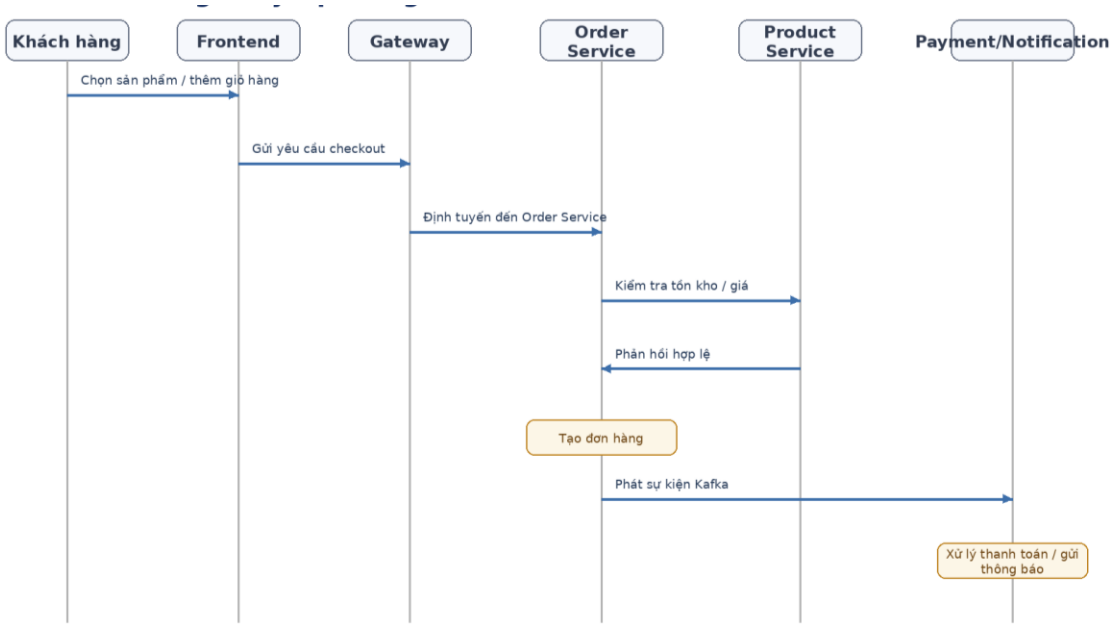
3.5. Vai trò từng service trong nghiệp vụ

Service	Vai trò chính	Phụ thuộc hạ tầng nổi bật
user-service	Quản lý tài khoản, xác thực, hồ sơ người dùng, phân quyền.	MongoDB, Gateway, Eureka
product-service	Quản lý sản phẩm, truy vấn danh mục, cache dữ liệu sản phẩm.	MongoDB, Redis, Kafka, Eureka, Zipkin
order-service	Tạo đơn hàng, kiểm tra điều kiện nghiệp vụ, theo dõi trạng thái.	MongoDB, Redis, Kafka, product-service, Eureka
payment-service	Xử lý thanh toán hoặc đóng vai trò abstraction cho thanh toán.	Gateway, Eureka; dễ tích hợp đối tác
notification-service	Nhận sự kiện để gửi email/thông báo sau giao dịch.	Kafka
api-gateway	Định tuyến request, gom điểm truy cập, xử lý CORS.	Eureka
eureka-server	Đăng ký/tìm kiếm dịch vụ trong môi trường động.	Hạ tầng cốt lõi

3.6. Luồng nghiệp vụ chính của hệ thống

Một luồng nghiệp vụ điển hình bắt đầu từ người dùng tương tác với frontend để xem sản phẩm, thêm sản phẩm vào giỏ và thực hiện checkout. Frontend gửi request đến API Gateway. Gateway định tuyến request đến order-service. Order-service nhiều khả năng sẽ xác nhận thông tin đơn hàng, lấy hoặc kiểm tra dữ liệu liên quan đến sản phẩm, sau đó lưu đơn hàng vào MongoDB. Khi đơn được tạo thành công, order-service có thể phát sự kiện qua Kafka để payment-service hoặc notification-service tiếp tục xử lý các bước hậu kiểm như thanh toán, gửi email, cập nhật dashboard hoặc ghi log.

Luồng này cho thấy sự kết hợp giữa gọi đồng bộ và bất đồng bộ. Phần cần phản hồi ngay cho người dùng như tạo đơn hàng thường là đồng bộ; phần hậu xử lý như gửi thông báo phù hợp hơn với bất đồng bộ để giảm thời gian chờ trên giao diện.



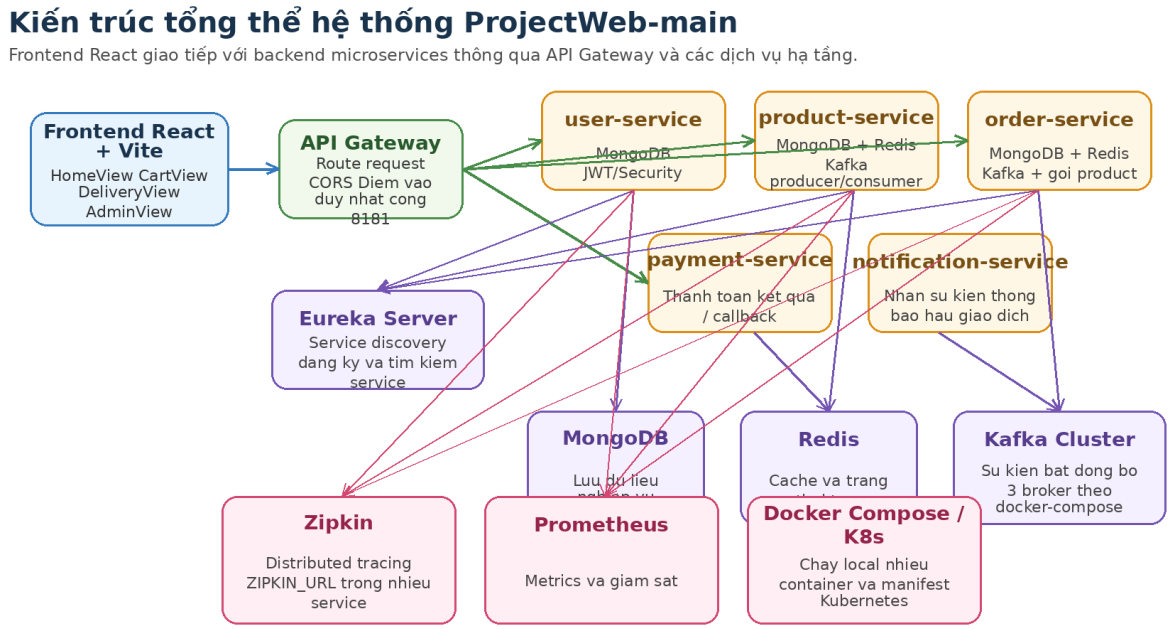
Hình 3. Luồng xử lý đặt hàng điển hình

3.7. Nhận xét về mức độ hoàn thiện của dự án

So với nhiều bài tập chỉ dừng ở mức chia controller thành nhiều package, ProjectWeb-main thể hiện mức độ hoàn thiện tốt hơn nhờ có đủ những thành phần thường thấy trong hệ microservices hiện đại. Đáng chú ý là sự hiện diện đồng thời của Gateway, Eureka, Kafka, Redis, Docker Compose, Kubernetes, Zipkin, Prometheus và CI/CD. Điều này giúp sinh viên có môi trường để học không chỉ về lập trình service mà còn về tích hợp, vận hành và triển khai.

Dù vậy, README backend còn có điểm chưa đồng nhất với pom và docker-compose, ví dụ README nêu PostgreSQL và Java 17 trong khi cấu hình đọc được cho thấy MongoDB, Redis và parent pom dùng Java 21. Đây là dấu hiệu cho thấy tài liệu dự án cần được cập nhật đồng bộ hơn để phản ánh chính xác hệ thống hiện tại.

CHƯƠNG 4. THIẾT KẾ KIẾN TRÚC MICROSERVICES CỦA HỆ THỐNG

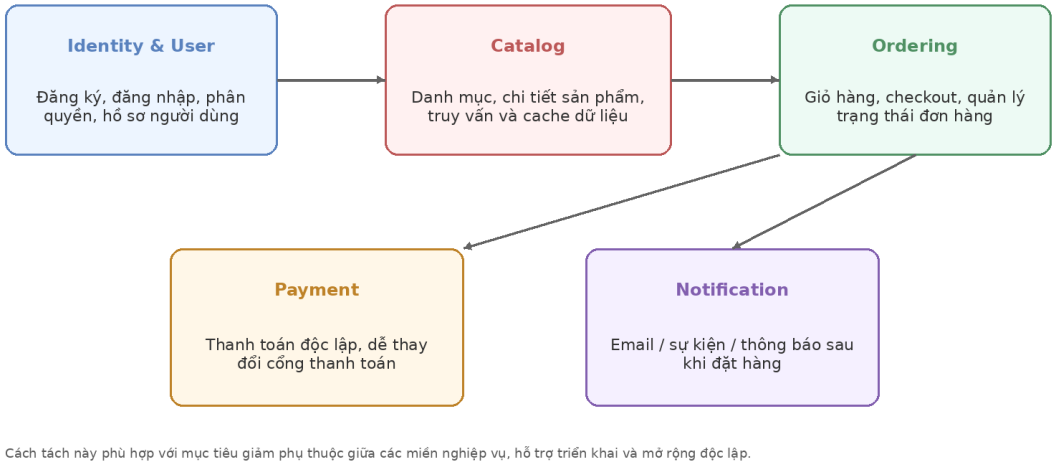


Hình 4 . Kiến trúc tổng thể hệ thống ProjectWeb-main

4.1. Nguyên tắc phân tách miền nghiệp vụ

Một dự án microservices tốt cần phân tách service theo miền nghiệp vụ chứ không chỉ theo lớp kỹ thuật. Trong ProjectWeb-main, việc tách user-service, product-service, order-service, payment-service và notification-service là hợp lý vì mỗi service đại diện cho một bounded context đủ rõ. Bằng cách này, nhóm phát triển có thể tập trung tối ưu logic riêng của từng miền mà không làm rối toàn bộ hệ thống.

User là miền về danh tính và quyền truy cập; Product là miền về catalog; Order xử lý vòng đời đơn hàng; Payment là miền giao dịch tài chính; Notification xử lý giao tiếp hậu sự kiện. Đây là mô hình phân tách phổ biến trong thương mại điện tử và cũng khá gần với thực tiễn doanh nghiệp.



Hình 5. Phân tách miền nghiệp vụ theo bounded context

4.2. API Gateway

API Gateway là thành phần cực kỳ quan trọng trong dự án. Thay vì để frontend gọi trực tiếp đến từng service với nhiều cổng và URL khác nhau, gateway đóng vai trò cửa vào duy nhất. Cách tiếp cận này mang lại một số lợi ích: che giấu topology nội bộ, đơn giản hóa cấu hình client, tập trung kiểm soát CORS, và tạo nền tảng cho các chức năng nâng cao như xác thực tập trung, rate limiting, logging và quan sát lưu lượng.

Trong docker-compose, api-gateway được map cổng 8181, phụ thuộc eureka-server và được cấu hình FRONTEND_ALLOWED_ORIGINS, cho thấy gateway là tầng bảo vệ và điều phối chính giữa frontend và các service nội bộ.

4.3. Service Discovery với Eureka

Trong môi trường microservices, địa chỉ service có thể thay đổi khi triển khai lại, scale ngang hoặc chuyển môi trường. Nếu các service gọi nhau bằng địa chỉ cứng, hệ thống sẽ khó linh hoạt. Eureka giải quyết vấn đề này bằng cơ chế đăng ký và khám phá dịch vụ. Mỗi service khi khởi động sẽ đăng ký với eureka-server; gateway hoặc service khác có thể tra cứu để định tuyến đúng đích.

Sự hiện diện của Eureka cho thấy dự án đã vượt khỏi mức cấu hình tĩnh đơn giản. Đây là bước quan trọng nếu muốn tiến tới mô hình tự scale hoặc triển khai trên Kubernetes hoặc cluster.

4.4. Giao tiếp đồng bộ và bất đồng bộ

Dựa trên thành phần hạ tầng, hệ thống sử dụng cả hai kiểu giao tiếp. Đồng bộ chủ yếu diễn ra qua HTTP/REST khi frontend gửi yêu cầu đọc hoặc ghi dữ liệu hoặc khi service cần phản hồi ngay. Bất đồng bộ được hỗ trợ bởi Kafka, phù hợp cho các sự kiện như: đơn hàng đã tạo, thanh toán đã xác nhận, gửi email hoặc cập nhật thông báo. Kết hợp hai kiểu giao tiếp này giúp hệ thống vừa đáp ứng nhanh ở những thao tác tương tác trực tiếp, vừa giảm coupling trong các công việc hậu xử lý.

Đây là mô hình rất phù hợp cho bài toán thương mại điện tử vì nhiều hành động sau checkout không nhất thiết phải chặn luồng phản hồi với người dùng.

Hình 6 . Chuỗi xử lý đặt hàng và hậu xử lý sự kiện

4.5. Quản lý dữ liệu

Docker-compose cho thấy MongoDB được dùng làm database dùng chung về hạ tầng cho nhiều service. Ở mức học thuật, cách này giúp triển khai nhanh. Tuy nhiên về nguyên tắc microservices, mỗi service nên sở hữu dữ liệu của riêng mình hoặc ít nhất là schema hoặc tập collection riêng, tránh việc nhiều service truy cập chéo trực tiếp vào cùng một cấu trúc dữ liệu. Nếu tiến xa hơn, hệ thống có thể tách database theo từng service hoặc áp dụng pattern database-per-service để tăng tính độc lập.

Redis được dùng cho product-service và order-service, nhiều khả năng nhằm cache dữ liệu truy vấn hoặc lưu trạng thái tạm thời. Đây là lựa chọn hợp lý vì catalog và đơn hàng là hai miền có nhu cầu đọc thường xuyên và cần phản hồi nhanh.

4.6. Bảo mật và xác thực

Từ parent pom có thể nhận thấy hệ thống sử dụng Spring Security và thư viện JWT. Điều này cho phép suy luận rằng xác thực đăng nhập được thực hiện thông qua token, một lựa chọn phổ biến trong các hệ client-server hiện đại. Với mô hình này, user-service hoặc một auth module sẽ phát hành token sau khi người dùng đăng nhập thành công; gateway hoặc từng service sau đó kiểm tra token để xác định danh tính và quyền truy cập.

Trong microservices, việc đặt một phần kiểm tra bảo mật ở gateway giúp giảm lặp logic và kiểm soát đầu vào tốt hơn. Tuy nhiên các service quan trọng vẫn nên tự kiểm tra claims hoặc quyền cần thiết để tránh phụ thuộc hoàn toàn vào gateway.

4.7. Quan sát hệ thống và truy vết

Zipkin và Prometheus là hai chỉ dấu cho thấy hệ thống chú trọng khả năng quan sát. Zipkin giúp truy vết một request đi qua gateway, service nghiệp vụ, database hoặc message broker như thế nào. Điều này đặc biệt cần thiết khi lỗi không xuất hiện ở một nơi duy nhất. Prometheus hỗ trợ thu thập metric như số request, độ trễ, tình trạng health check, từ đó phục vụ giám sát và cảnh báo.

Trong một đồ án học thuật, việc bổ sung các công cụ quan sát như vậy làm tăng giá trị hệ thống, vì sinh viên được tiếp cận với tư duy vận hành thay vì chỉ hoàn thành chức năng.

CHƯƠNG 5. TRIỂN KHAI, VẬN HÀNH VÀ CÔNG NGHỆ SỬ DỤNG

5.1. Stack công nghệ của dự án

Lớp	Công nghệ	Vai trò
Frontend	React, Vite, Tailwind CSS, Axios	Xây dựng giao diện web, điều hướng, gọi API.
Backend	Java, Spring Boot, Spring Cloud	Phát triển microservice và hạ tầng service.
Discovery	Eureka Server	Đăng ký và khám phá service.
Gateway	Spring Cloud Gateway	Định tuyến, bảo vệ và gom đầu vào.

Database	MongoDB	Lưu trữ dữ liệu chính.
Cache	Redis	Tăng tốc truy vấn, lưu trạng thái tạm.
Messaging	Apache Kafka	Truyền sự kiện bất đồng bộ.
Container	Docker, Docker Compose	Đóng gói và chạy local nhiều service.
Orchestration	Kubernetes / Minikube	Triển khai theo cụm.
Observability	Zipkin, Prometheus	Tracing và monitoring.
CI/CD	GitHub Actions	Tự động hóa build và deploy.

5.2. Triển khai local với Docker Compose

Docker Compose của backend mô tả một môi trường local tương đối đầy đủ. Hệ thống khởi tạo Redis, MongoDB, Prometheus, Zipkin, một cụm Kafka ba broker, Kafka UI, eureka-server, user-service, product-service, order-service, notification-service, payment-service và api-gateway. Cách tổ chức này cho phép người phát triển mô phỏng được môi trường phân tán ngay trên máy cá nhân.

Một điểm đáng chú ý là Kafka được cấu hình theo mô hình KRaft với ba broker, điều này nâng mức thực hành lên cao hơn so với việc chỉ chạy một broker đơn giản. Dù trong môi trường học tập chưa chắc cần cluster đầy đủ, nhưng cấu hình này giúp người học tiếp cận gần hơn với cách hệ thống production được tổ chức.

Thành phần	Cổng/đặc điểm	Ý nghĩa vận hành
redis	6379	Cache dùng chung cho product-service và order-service.
mongo	27017	Kho dữ liệu chính cho nhiều service nghiệp vụ.
zipkin	9411	Quan sát trace xuyên service.
prometheus	9090	Thu thập và giám sát metric.
kafka-1 / kafka-2 / kafka-3	9092 / 9094 / 9096	Cụm broker để thử nghiệm event-driven.
kafka-ui	8080	Theo dõi topic, message và cluster Kafka.
eureka-server	8761	Service discovery.
api-gateway	8181	Điểm public chính cho frontend.
user / product / order / notification /	8081-8085	Các service nghiệp vụ.

payment		

5.3. CI/CD và container hóa

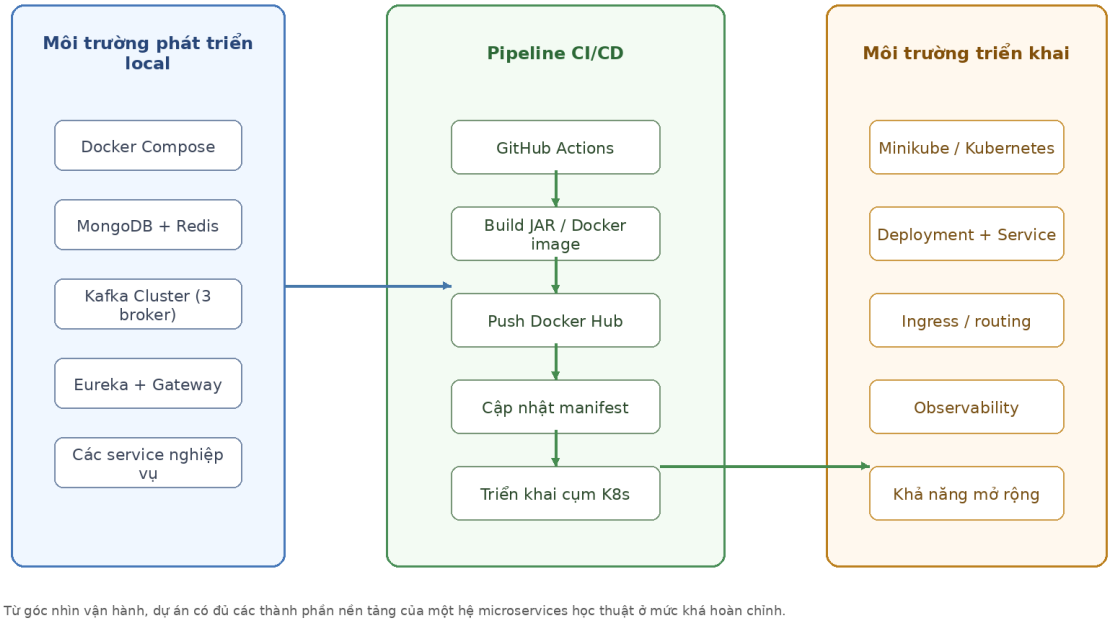
README backend nêu rõ mỗi service có Dockerfile riêng và yêu cầu Docker Hub secrets trong GitHub repo. Điều này cho thấy pipeline CI/CD được thiết kế để build image cho từng service, đẩy lên registry rồi triển khai lại lên cụm. Việc tách image theo service là đúng với tinh thần microservices, vì mỗi service có thể nâng phiên bản độc lập.

Sự tồn tại của thư mục .github/workflows cho thấy dự án hướng tới tự động hóa quy trình build, test và deploy. Trong môi trường học tập, đây là một điểm cộng lớn vì nó mô phỏng chính xác quy trình DevOps hiện đại.

5.4. Triển khai trên Kubernetes

Thư mục k8s cùng tệp kubeconfig-ci.yaml cho thấy hệ thống đã được chuẩn bị cho triển khai lên Kubernetes, ít nhất là qua Minikube hoặc một cụm thử nghiệm. Khi đưa microservices lên Kubernetes, lợi ích lớn nhất là khả năng quản lý lifecycle container, rolling update, restart tự động khi có lỗi và mở rộng số lượng pod. Đồng thời, các tài nguyên như Deployment, Service, Ingress giúp tách rõ mô hình nội bộ và mô hình công bố ra bên ngoài.

Với một dự án thương mại điện tử, Kubernetes cũng tạo nền tảng để tiến xa hơn đến autoscaling, cấu hình secret/configmap và chuẩn hóa hạ tầng đa môi trường.

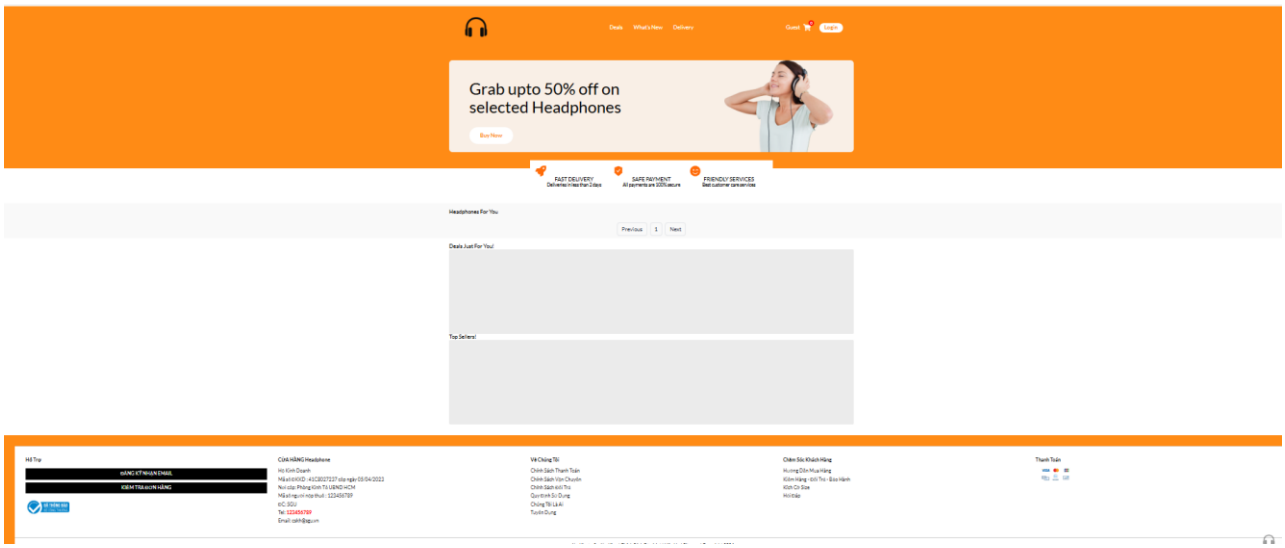


Hình 7. Mô hình triển khai và vận hành

5.5. Tính nhất quán tài liệu và cấu hình

Trong quá trình phân tích, có thể nhận thấy một số chỗ tài liệu chưa đồng nhất hoàn toàn với cấu hình hiện tại. README backend nêu PostgreSQL và Java 17, nhưng pom tổng cho thấy parent project dùng Java 21 và docker-compose thể hiện MongoDB, Redis. Điều này không làm mất giá trị của hệ thống, nhưng là một vấn đề quản trị tài liệu cần được lưu ý.

Đối với báo cáo kỹ thuật, điểm này rất quan trọng vì tài liệu không đồng bộ sẽ khiến người khác khó chạy dự án hoặc hiểu sai kiến trúc. Một hệ thống microservices càng lớn thì tài liệu càng cần được cập nhật song song với mã nguồn.

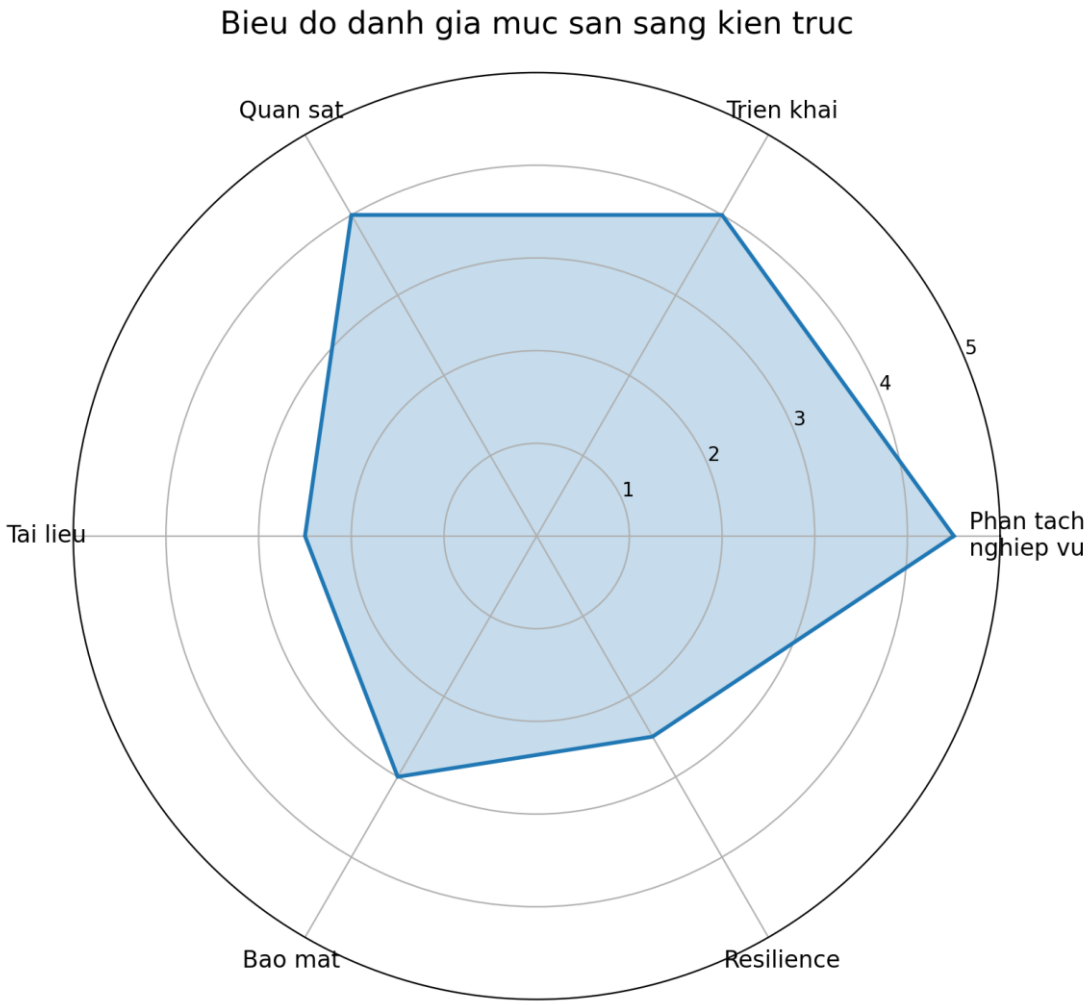


Hình 8: trang chủ của trang web

CHƯƠNG 6. ĐÁNH GIÁ HỆ THỐNG

6.1. Ưu điểm

- Hệ thống được phân tách service tương đối hợp lý theo miền nghiệp vụ.
- Có đủ các thành phần hạ tầng cốt lõi của microservices: gateway, discovery, cache, broker, tracing, monitoring.
- Hỗ trợ cả môi trường local và định hướng triển khai cluster.
- Frontend và backend được tách rõ, phù hợp cho phát triển song song.
- Cấu trúc multi-module Maven giúp quản lý phụ thuộc rõ ràng.
- Việc xuất hiện Kafka và notification-service cho thấy hệ thống đã bắt đầu áp dụng event-driven architecture.



Hình 9. Biểu đồ đánh giá mức sẵn sàng kiến trúc

6.2. Hạn chế

- Tài liệu README chưa hoàn toàn đồng bộ với cấu hình thực tế.
- Chưa thấy rõ chiến lược database-per-service ở mức tài liệu.
- Chưa thể hiện rõ cơ chế fault tolerance như circuit breaker, retry, rate limiting.
- Chưa có thông tin chi tiết về chiến lược kiểm thử tích hợp, contract test hoặc bảo mật end-to-end.
- Chưa thể hiện rõ versioning của API và quản lý schema sự kiện Kafka.

6.3. Đánh giá theo tiêu chí kiến trúc

Tiêu chí	Nhận xét	Mức độ
Phân tách	Tách user, product, order, payment,	Tốt

nghiệp vụ	notification tương đối đúng ranh giới.	
Giao tiếp service	Có đồng bộ qua gateway và bất đồng bộ qua Kafka.	Tốt
Khả năng mở rộng	Có thể scale từng service và triển khai qua K8s.	Tốt
Quan sát hệ thống	Có Zipkin, Prometheus.	Tốt
Tài liệu	Còn chỗ chưa nhất quán.	Trung bình
Mức độ production-ready	Có nền tảng tốt nhưng cần bổ sung resiliency và governance.	Khá

6.4. Phân tích rủi ro kỹ thuật

Trong microservices, rủi ro lớn nhất thường không nằm ở một service đơn lẻ mà ở mối liên kết giữa các service. Nếu gateway cấu hình chưa chặt, client có thể gặp lỗi định tuyến hoặc CORS. Nếu service discovery không ổn định, việc gọi nội bộ sẽ gián đoạn. Nếu Kafka hoặc Redis không được giám sát đúng cách, hệ thống có thể phát sinh lỗi khó truy vết. Dự án đã có các thành phần nền tảng để giảm rủi ro, nhưng cần bổ sung dashboard và cơ chế cảnh báo cụ thể để hỗ trợ vận hành.

Ngoài ra, khi chia nhỏ service, việc quản lý version API, schema dữ liệu và dữ liệu dùng chung trở thành thách thức. Đây là lý do microservices không chỉ là vấn đề code mà còn là vấn đề quản trị kỹ thuật tổng thể.

CHƯƠNG 7. HƯỚNG PHÁT TRIỂN

7.1. Hoàn thiện tài liệu kỹ thuật

Ưu tiên đầu tiên là cập nhật README và tài liệu kiến trúc để phản ánh chính xác stack hiện tại, cách chạy từng service, sơ đồ service, cổng truy cập, biến môi trường, cách seed dữ liệu và mô tả luồng nghiệp vụ. Một dự án microservices càng nhiều thành phần thì tài liệu càng đóng vai trò quan trọng.

7.2. Tăng cường bảo mật

Hệ thống có thể bổ sung một auth-server tách riêng hoặc chuẩn hóa user-service thành identity provider rõ ràng hơn. Ở gateway nên triển khai rõ ràng xác thực JWT, giới hạn truy

cập theo route, thêm refresh token, quản lý vai trò user/admin và log truy cập. Đồng thời nên lưu secret qua environment hoặc Kubernetes Secret thay vì hard-code hoặc để phân tán không kiểm soát.

7.3. Tăng độ bền vững của hệ thống

Để hệ thống tiến gần hơn tới production, cần bổ sung các pattern resilience như timeout, retry có kiểm soát, circuit breaker, bulkhead và idempotency cho các thao tác quan trọng. Với order-service và payment-service, idempotency là rất cần thiết để tránh tạo trùng giao dịch khi người dùng nhấn lại hoặc khi request bị lặp.

7.4. Cải tiến dữ liệu và hiệu năng

Có thể xem xét tách database theo service, áp dụng event sourcing một phần cho các sự kiện đơn hàng hoặc thêm read model phục vụ dashboard. Redis có thể được dùng bài bản hơn cho cache catalog, hot products, session và token blacklist. Product-service có thể bổ sung chiến lược invalidation cache khi sản phẩm thay đổi.

7.5. Bổ sung kiểm thử và quản trị API

Dự án nên thêm các tầng kiểm thử như unit test, integration test, API test, contract test giữa gateway và các service, cũng như test hiệu năng cho product-service và order-service. Đồng thời cần chuẩn hóa tài liệu API bằng OpenAPI hoặc Swagger để frontend và backend dễ phối hợp hơn.

KẾT LUẬN

Qua quá trình phân tích, có thể khẳng định ProjectWeb-main là một dự án học tập có giá trị cao trong việc nghiên cứu kiến trúc microservices. Hệ thống không chỉ chia thành các service nghiệp vụ mà còn được bổ sung đầy đủ nhiều thành phần hạ tầng quan trọng như Gateway, Eureka, Kafka, Redis, MongoDB, Zipkin, Prometheus, Docker Compose, Kubernetes và CI/CD. Điều này giúp dự án phản ánh tương đối đầy đủ vòng đời của một hệ thống phân tán hiện đại.

Từ góc độ kiến trúc, cách phân tách service là hợp lý và phù hợp với bài toán thương mại điện tử. Từ góc độ vận hành, hệ thống đã có nền tảng tốt nhưng vẫn cần hoàn thiện tài liệu, đồng bộ cấu hình, tăng cường bảo mật, bổ sung cơ chế chống lỗi lan truyền và chuẩn hóa dữ liệu. Nếu tiếp tục được phát triển, đây có thể trở thành một mô hình tham khảo tốt cho việc học và thực hành microservices trong môi trường đại học.

TÀI LIỆU THAM KHẢO

- Repository GitHub: Duck1702/CCNLTHD, thư mục BaiTap6/ProjectWeb-main.
- README frontend trong FE/react-e-commerce.
- README backend, pom.xml tổng và docker-compose.yml trong BE.
- Tài liệu Spring Boot và Spring Cloud Gateway.
- Tài liệu Spring Cloud Netflix Eureka.
- Tài liệu Apache Kafka, Redis, MongoDB, Docker và Kubernetes.